

PYTHON OOP — INDIVIDUAL ASSIGNMENT

Student: Pawan

Project: Employee Payroll Management System

Scope: First 11 OOP concepts

1. Problem Statement

Create a company payroll application that stores employee details and monthly salary information. The system should calculate salary information, display employee records and maintain the total number of employees created.

2. What You Need to Achieve

- Create at least 3 independent objects and prove their data is separate.
- Use `__init__()` to initialize object data and self inside instance methods.
- Implement at least 4 useful instance methods for the business operation.
- Use instance variables for object-specific data.
- Use a class variable to maintain a common object count.
- Use `@classmethod` with `cls` to display the class-level count.
- Use `@staticmethod` for validation or a utility operation that needs neither self nor `cls`.
- Handle at least 3 invalid/error cases cleanly.
- Display clear, readable output.

3. Mandatory Concepts

No.	Concept
1	Class
2	Object / Instance
3	Attribute
4	Method
5	<code>__init__()</code>
6	<code>self</code>
7	Instance variable
8	Class variable
9	Class method (<code>@classmethod</code>)
10	Static method (<code>@staticmethod</code>)

4. Instructions

- Keep the project limited to the concepts taught so far. Do not use inheritance, polymorphism, abstraction, composition or external databases.
- Business data and behavior must be organized inside your class; do not build the whole project as unrelated functions.
- Use meaningful class, variable and method names.
- Add comments showing where each mandatory OOP concept is used.
- Create a Python file and a `README.md`.
- Test normal cases plus at least 3 invalid cases.

5. Minimum Demonstration

- Create 3 objects and display their initial details.
- Perform the main business operation(s) and show updated data.
- Call the class method and display the total object count.
- Call the static method and demonstrate valid and invalid input.
- Show at least 3 error/validation scenarios.

6. Submission

- Python source file (.py)
- README.md containing problem statement, OOP concepts used and sample output
- Execution screenshot
- At least 3 test cases with expected and actual results
- Be ready to explain every class, object, attribute, method, `__init__`, `self`, instance variable, class variable, class method and static method.

7. Evaluation

- Class and object design — 15 marks
- `__init__`, `self` and instance members — 20 marks
- Instance methods and business logic — 20 marks
- Class variable and class method — 15 marks
- Static method and validation — 10 marks
- Testing, readability and README — 20 marks

Teacher Note

This assignment is intentionally restricted to the first 11 concepts. The purpose is to make students strong in objects, state, behavior and class-level utilities before moving to encapsulation, inheritance, polymorphism, abstraction and composition.